CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

joinSpheres-returnByValue.cpp

```
11 /*
12  * Creates a new sphere that contains the exact volume
13  * of the volume of the two input spheres.
14  */
15 Sphere joinSpheres(const Sphere &s1, const Sphere &s2) {
16   double totalVolume = s1.getVolume() + s2.getVolume();
17
18   double newRadius = std::pow(
19     (3.0 * totalVolume) / (4.0 * 3.141592654),
20     1.0/3.0
21   );
22
23   return Sphere(newRadius);
24 }
```

```
28 int main() {
29   Sphere *s1 = new Sphere(4);
30   Sphere *s2 = new Sphere(5);
31
32   Sphere s3 = joinSpheres(*s1, *s2);
33
34   delete s1; s1 = NULL;
35   delete s2; s2 = NULL;
36
37   return 0;
28 }
```

joinSpheres-returnByPointer.cpp

```
11 /*
12  * Creates a new sphere that contains the exact volume
13  * of the volume of the two input spheres.
14  */
15 Sphere *joinSpheres(const Sphere &s1, const Sphere &s2) {
16   double totalVolume = s1.getVolume() + s2.getVolume();
17
18   double newRadius = std::pow(
19     (3.0 * totalVolume) / (4.0 * 3.141592654),
20     1.0/3.0
21   );
22
23   return
       new Sphere(newRadius);
24 }
```

```
28 int main() {
29   Sphere *s1 = new Sphere(4);
30   Sphere *s2 = new Sphere(5);
31
32   Sphere *s3 = joinSpheres(*s1, *s2);
33
34   delete s1; s1 = NULL;
35   delete s2; s2 = NULL;
36   delete s3; s3 = NULL;
37   return 0;
28 }
```

joinSpheres-returnByReference.cpp

```
11 /*
12  * Creates a new sphere that contains the exact volume
13  * of the volume of the two input spheres.
14  */
15 Sphere & joinSpheres(const Sphere &s1, const Sphere &s2) {
16   double totalVolume = s1.getVolume() + s2.getVolume();
17
18   double newRadius = std::pow(
19     (3.0 * totalVolume) / (4.0 * 3.141592654),
20     1.0/3.0
21   );
22
23
24
25   Sphere *result =
        new Sphere(newRadius);
26   return *result;
27 }
```

```
28 int main() {
29   Sphere *s1 = new Sphere(4);
30   Sphere *s2 = new Sphere(5);
31
32   Sphere s3 = joinSpheres(*s1, *s2);
33
34   delete s1; s1 = NULL;
35   delete s2; s2 = NULL;
36
37   return 0;
28 }
```

joinSpheres-returnByReference2.cpp

```
11 /*
12  * Creates a new sphere that contains the exact volume
13  * of the volume of the two input spheres.
14  */
15 Sphere & joinSpheres(const Sphere &s1, const Sphere &s2) {
16   double totalVolume = s1.getVolume() + s2.getVolume();
17
18   double newRadius = std::pow(
19     (3.0 * totalVolume) / (4.0 * 3.141592654),
20     1.0/3.0
21   );
22
23
24
25   Sphere result(newRadius);
26   return result;
27 }
```

```
28 int main() {
29   Sphere *s1 = new Sphere(4);
30   Sphere *s2 = new Sphere(5);
31
32   Sphere s3 = joinSpheres(*s1, *s2);
33
34   delete s1; s1 = NULL;
35   delete s2; s2 = NULL;
36
37   return 0;
28 }
```

addSpheres.cpp

```
1 #include "sphere.h"
2
3 int main() {
4   cs225::Sphere s1(3), s2(4);
5   cs225::Sphere s3 = s1 + s2;
6   return 0;
7 }
```

```
addSpheres.cpp: In function 'int main()':
addSpheres.cpp:7:25: error: no match for 'operator+' (operand types are 'cs225::
Sphere' and 'cs225::Sphere')
   cs225::Sphere s3 = s1 + s2;
                      ~~~^~~~
```

Operators that can be overloaded in C++	
Arithmetic	+ - * / % ++ --
Bitwise	& | ^ ~ << >>
Assignment	=
Comparison	== != > < >= <=
Logical	! && ||
Other	[] () ->

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
11
12
13
14
…         // ...
26      private:
27        double r_;
28
29    };
30  }
31
32  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5
 6
 7
 8
 9
10
11
12
13
14
15
16
17
18
19
20
…       // ...
    }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
11
12        Sphere operator+(const Sphere& s);
13
14
…         // ...
26      private:
27        double r_;
28
29    };
30  }
31
32  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5    Sphere Sphere::operator+(
 6                const Sphere& s) {
 7
 8      return joinSpheres(this, &s);
 9
10    }
11
12
13
14
15
16
17
18
19
20
…     // ...
  }
```

addSpheres.cpp

```
1  #include "sphere.h"
2
3  int main() {
4    cs225::Sphere s1(3), s2(4);
5    cs225::Sphere s3 = s1 + s2;
6    return 0;
7  }
```

One Very Special Operator

Definition Syntax (.h):

```
Sphere& operator=(const Sphere& s)
```

Implementation Syntax (.cpp):

```
Sphere& Sphere::operator=(const Sphere& s)
```

Assignment Operator

Similar to Copy Constructor:

- It copies the contents of another object
- It has an automatic default
- If you do not define it, you will get one automatically created for you, which will do the same shallow copy as the default copy constructor

Different from Copy Constructor:

- The copy constructor is called when a new object is created whereas the assignment operator is called on an existing object
- So, if the object was *never constructed* in the first place, the copy constructor will be copied
- But when we are assigning to an *existing* object, its assignment operator is called

What constructors and operators are called?

```
1  Sphere s1, s2;
2
3  s2 = s1;
4
5  Sphere s3 = s1;
6
7  Sphere *s4 = &s3;
8
9  Sphere &s5 = s2;
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
11        Sphere& operator=(
12              const Sphere& s);
13
14
 …        // ...
26      private:
27        double r_;
28
29    };
30  }
31
32  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5    Sphere& Sphere::operator=(
 6                const Sphere& s) {
 7       r_ = s.r_;
 8       return *this;
 9    }
10
11
12
13
14
15
16
17
18
19
20
 …     // ...
   }
```

Destructor

- The destructor is the last function that is called on an existing instance before it disappears
- The purpose is to clean up resources
- It will never be called manually, it is always called automatically when
  - Your object is deleted from the heap
  - Or goes out of scope on the stack
- The default is provided

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
11        ~Shpere();
12
13
14
 …        // ...
26      private:
27        double r_;
28
29    };
30  }
31
32  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5
 6   Sphere::~Shpere() {}
 7
 8
 9
10
11
12
13
14
15
16
17
18
19
20
 …     // ...
   }
```

Assignment Operator

	Copies an object	Destroys an object
Copy constructor		
Copy Assignment operator		
Destructor		

The “Rule of Three”

If it is necessary to define **any one** of these three functions in a class, it will be necessary to define **all three** of these functions:

- **Copy constructor**
- **Assignment operator**
- **The destructor**

Towards a more advanced Sphere…

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
…
26        // ...
27      private:
28        double r_;
29        std::string *props_;
30        unsigned props_ct;
31        unsigned props_max;
32
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5
 6
 7
 8
 9
10
11
12
13
14
15
16
17
18
19
20
…       // ...
    }
```

redSphere-main.cpp

```
 1 // Create a Sphere with various properties, like color,
 2 // texture, and others.
 3 #include "Sphere.h"
 4
 5 int main() {
 6   cs225::Sphere s(10);
 7   s.addProperty("Red");
 8   s.addProperty(“Rubber");
 9   return 0;
10 }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10
…
26        // ...
27      private:
28        double r_;
29        std::string *props_;
30        unsigned props_ct;
31        unsigned props_max;
32
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5
 6    Sphere::Sphere(double r) {
 7       r_ = r;
 8       propos_max = 5;
 9       props_ct = 0;
10       props_ = new std::string[5];
11    }
12
13
14
15
16
17
18
19
20
…      // ...
   }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10        void addProperty(
…             const ssd::string &s);
26
27        // ...
28      private:
29        double r_;
30        std::string *props_;
31        unsigned props_ct;
32        unsigned props_max;
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5    // needs error checking
 6    void Sphere::addProperty(
 7          const ssd::string &s) {
 8
 9       props_[props_ct_] = s;
10       props_ct_++;
11    }
12
13
14
15
16
17
18
19
20
…     // ...
    }
```

redSphere-main.cpp

```
 1 // Create a Sphere with various properties, like color,
 2 // texture, and others.
 3 #include "Sphere.h"
 4
 5 int main() {
 6   cs225::Sphere s1(10);
 7   s1.addProperty("Red");
 8   s1.addProperty(“Rubber");
 9   cs225::Sphere s2;
10   s2 = s1;
11   s2.addProperty(“Purple");
12   s1.addProperty(“Green");
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10        void addProperty(
…             const ssd::string &s);
26
27       // ...
28      private:
29        double r_;
30        std::string *props_;
31        unsigned props_ct;
32        unsigned props_max;
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5   Sphere::Sphere(const Sphere &s){
 6     r_ = s.r_;
 7     props_max_ = s.props_max_;
 8     props_ct_ = s.props_ct_;
 9     props_ = new std::string[5];
10
11     for (int i = 0; i < props_ct_; i++)
12        props_[i] = s.props_[i];
13   }
14
15
16
17
18
19
20
…      // ...
     }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10        void addProperty(
…           const ssd::string &s);
26       Sphere& Sphere::operator=(
27                const Sphere& s);
28      private:
29        double r_;
30        std::string *props_;
31        unsigned props_ct;
32        unsigned props_max;
33     void copy_(const Sphere &s);
34    };
35  }
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4   void Sphere::copy_(const Sphere &s) {
 5     r_ = s.r_;
 6     props_max_ = s.props_max_;
 7     props_ct_ = s.props_ct_;
 8     props_ = new std::string[5];
 9     for (int i = 0; i < props_ct_; i++)
10        props_[i] = s.props_[i];
11   }
12   Sphere::Sphere(const Sphere &s) {
13     copy_(s);
14   }
15   Sphere& Sphere::operator=(
16               const Sphere& s) {
17     delete [] props_; props_ = NULL;
18     copy_(s);
19     return *this;
20   }
…    // ...
     }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10        ~Sphere();
 …
26        // ...
27      private:
28        double r_;
29        std::string *props_;
30        ...
31
32
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5    Sphere::~Sphere() {
 6      delete [] props_;
 7    }
 8
 9
10
11
12
13
14
15
16
17
18
19
20
 …      // ...
   }
```

sphere.h

```
 1  #ifndef SPHERE_H
 2  #define SPHERE_H
 3
 4  namespace cs225 {
 5    class Sphere {
 6      public:
 7        Sphere();
 8        Sphere(double r);
 9        Sphere(const Sphere &s);
10        ~Sphere();
 …
26        // ...
27      private:
28        double r_;
29     void copy_(const Sphere &s);
30        void destroy_();
31        ...
32
33    };
34  }
35
36  #endif
```

sphere.cpp

```
 1  #include "sphere.h"
 2
 3  namespace cs225 {
 4
 5    Sphere::~Sphere() {
 6      destroy_();
 7    }
 8    void Sphere::destroy() {
 9      delete [] props_;
10      props_ = nulprt;
11    }
12
13    Sphere& Sphere::operator=(
14                const Sphere& s) {
15      if (this != &s) {
16        destroy_();
17        copy_(s);
18      }
19      return *this;
20   }
 …    // ...
     }
```

The “Rule of Five”

If it is necessary to define **any one** of these three functions in a class, it will be necessary to define **all five** of these functions:

- **Copy constructor**
- **Assignment operator**
- **The destructor**

- **Move constructor**
- **Move assignment operator**

Move Semantics

Rvalue

A temporary object (e.g., Sphere(3)), bindable only to T&&.

Move Constructor (Steals resources from a temporary instead of copying)

```
Sphere(const Sphere&& s);
```

Move Assignment (Releases current resources, then takes ownership from s)

```
Sphere & operator=(const Sphere&& s);
```

The “Rule of Zero”

Corollary to Rule of Five

Classes that declare custom destructors, copy/move constructors or copy/move assignment operators should deal *exclusively* with ownership. Other classes should not declare custom destructors, copy/move constructors or copy/move assignment operators.

–Scott Meyers

CS 225 – Things To Be Doing

Quiz 1 was yesterday

At 19:00 in your discussion section room

50-minute quiz

More Info: Blackboard

lab_intro – Due this Sunday

Make sure to turn in lab_intro by Sunday

Work on MP1 – Due Monday after winter break

Do not delay working on it.